CS102 Week 4 Notes and Study Guide

Table of Contents

What is inside

- Week 4 overview: searching, search targets, search spaces, comparison steps, linear search, binary search, sorting, and choosing efficient strategies.
- Lesson 1 - Why Searching Matters
- Lesson 2 - How Searching Works with Simple Examples
- Lesson 3 - Linear Search with Small and Unsorted Data
- Lesson 4 - Binary Search with Step-by-Step Collections
- Lesson 5 - Comparing Linear and Binary Search
- Lesson 6 - Challenge: Searching
- Lesson 7 - Why Sorting Is Useful
- Lesson 8 - Sorting: Different Approaches
- Lesson 9 - Bubble Sort
- Lesson 10 - Insertion Sort
- Lesson 11 - Bubble vs. Insertion
- Lesson 12 - Challenge: Sorting
- Lesson 13 - Using Python Sorting Tools in Practice
- Lesson 14 - Advanced Sorting with Python Sorting Tools
- Lesson 15 - Challenge: Advanced Sorting
- Review tools, glossary, tracing practice, common mistakes, cheat sheet, and exam reference

Week 4 at a Glance

Week 4 moves from understanding algorithms to using algorithms for finding and organizing information. The first half focuses on searching: how a program looks through data, what it means to have a target, and why checking items one at a time can become slow as collections grow. The second half connects searching to sorting, because sorted data changes what strategies become possible.

The major story is trade-offs. Linear search is simple and works on unsorted data, but it may check every item. Binary search is much faster on large collections, but only works correctly when the data is sorted. Sorting itself takes work, so programmers choose based on the problem: how often the data changes, how often it is searched, and whether readability, speed, or simplicity matters most.

Memory hook: searching finds the target; sorting prepares the data; the best strategy depends on how the data is organized.

Week 4 Focus Questions

- What exact target am I trying to find?
- What search space am I allowed to inspect?
- Is the data sorted, unsorted, or changing often?
- Am I doing one quick search or searching the same data many times?
- Would sorting first make the work easier, or would it waste time?
- Does this problem care most about simplicity, speed, readability, or setup cost?

Week 4 Memory Hooks

- Searching finds the target.
- Sorting prepares the data.
- Linear search asks every item one by one.
- Binary search only works when the data is already sorted.
- The best choice depends on the situation, not the fanciest algorithm name.

Made with Goodnotes

Week 4 Master Framework

Core Thinking Skills

Skill	What it means	Why it matters
Search thinking	Identify what you are trying to find and where you are allowed to look.	Turns vague questions into a target plus a search space.
Comparison thinking	Check one candidate value against the target.	Every search depends on asking, "Is this the thing I want?"
Linear search	Check items one by one from start to finish.	Reliable for small or unsorted data, but may require many checks.
Binary search	Use the middle value to cut a sorted search space in half.	Much faster for large sorted data, but only works when order is guaranteed.
Sorting thinking	Arrange values into a useful order before searching or presenting them.	Organization can make later work easier, faster, and less chaotic.
Trade-off thinking	Compare simplicity, speed, prerequisites, and readability.	The best algorithm is the one that fits the actual situation, not just the flashiest one.

Search and Sort Decision Guide

Situation	Best first thought	Reason
Small list and no special order	Linear search	Simple, readable, and good enough.
Huge sorted list	Binary search	Repeated halving avoids checking everything.
Need to show items alphabetically or numerically	Sort first	Order makes the result easier for humans to use.
Data changes constantly	Be careful sorting early	Repeated sorting can waste work.
Searching the same data many times	Consider sorting or direct lookup	A little setup may save lots of repeated searching later.

Week 4 Big Picture

- Search problems ask: what target am I trying to find, and what data am I searching through?
- Linear search is the safest beginner pattern because it does not require sorted data.
- Binary search is powerful because each comparison removes about half of the remaining possibilities.
- Sorting is useful because ordered data is easier for both humans and algorithms to work with.
- Efficiency is not just fast or slow; it is about how work grows as input gets bigger.

Made with Goodnotes

Lesson 1 - Why Searching Matters

Big Idea

Searching means trying to locate a specific thing inside a group of information. In everyday life, searching might mean finding a contact, a photo, a file, or a song. In programming, searching means designing steps that help a computer find a target value inside a collection.

Full Teaching Explanation

The lesson starts with a phone contact example because it is familiar: if you need to call Zara, you could scroll through every contact, or you could use the search bar and jump much closer to the match. Both actions are searches, but they do not feel equally efficient. Scrolling checks possibilities one at a time. A search box uses a smarter path that reduces the amount of waiting.

That is the core reason searching matters in code. Programs often store large collections: contacts, emails, photos, product listings, grades, filenames, messages, usernames, or account records. If the program checks every possible item when it does not need to, the app can feel slow even when it eventually finds the correct result.

A useful way to phrase the problem is: "I know what I want, and I need to locate it in a bunch of stuff." The thing you want is the target. The bunch of stuff is the search space. The strategy is the set of steps the computer uses to move through that space.

Good searching affects user experience. If the first layer of search is slow, the whole app feels bad. If the first layer narrows results quickly, the human user can spend less time waiting and more time choosing the exact item they wanted.

Examples and Mental Models

Everyday situation	Search target	Search space	Why it matters
Find Zara in contacts	Zara	Contact list	The app should not make you scroll forever.
Find an invoice photo	Invoice screenshot	Photo library	The computer can filter quickly, but the human may still inspect results.
Find a song by title	Song name	Music library	Direct lookup feels much faster than opening random songs.
Find an email from Dr. Smith	Dr. Smith email	Inbox	A search feature saves time compared with checking messages one by one.

Out Loud Programmer Thinking

When I see a search problem, I should ask: What exactly am I looking for? Where am I allowed to look? Am I checking one item at a time, jumping directly to a known place, or using some other structure to narrow the results? If I can answer those questions, I can usually describe the search strategy before writing any Python.

Code Example

```
contacts = ["Alice", "Bob", "Charlie", "Zara"]

# Scroll-style search: check each contact one at a time.
for name in contacts:
    if name == "Zara":
        print("Found", name)

# Direct lookup style: jump straight to the known key.
phone_numbers = {"Zara": "555-1234"}
print(phone_numbers["Zara"])
```

The first pattern walks through the list one name at a time. The second pattern uses a key to jump straight to the information. The important idea is that different structures can make searching feel very different.

Made with Goodnotes

Lesson 1 - Why Searching Matters (continued)

Terms, Definitions, and Why They Are Called That

- Search: the process of looking for a specific item. It is called search because the answer is not immediately handed to you; the program must locate it.
- Target: the value you want to find. It is called the target because the search is aiming at that value.
- Search space: the collection of possible places where the target might be. It is the space the algorithm is allowed to inspect.
- Comparison: a check that asks whether the current item matches the target.
- Direct lookup: using a known key or location to retrieve information without checking every candidate first.

Common Mistakes and Helpful Tips

- Mistake: thinking all searches work the same way. Some searches scan one item at a time; others use structure to jump closer.
- Mistake: forgetting the human layer. A computer may find results quickly, but the user still has to recognize the right result.
- Tip: describe a search in one sentence: Find target inside search space.
- Tip: fewer unnecessary checks usually means a faster app and a happier user.

Review Checkpoint

- What is the search target in a contacts example?
- What is the search space in a photo-library example?
- Why does a search box usually feel faster than manual scrolling?
- How can a data structure make searching easier?

Key Takeaways

- Searching is one of the most common tasks real programs perform.
- A good search strategy reduces unnecessary work.
- The same user goal can be solved with slow scanning or faster direct access depending on the structure.
- User experience is affected by how quickly the first useful results appear.

Notes

Made with Goodnotes

Lesson 2 - How Searching Works with Simple Examples

Big Idea

A simple search has three parts: the search space, the search target, and comparison steps. The program looks through possible values, compares each candidate to the target, and stops when it finds the target or runs out of places to look.

Full Teaching Explanation

The lesson uses the lost-key example because it shows the search pattern clearly. You check the first pocket: no key. You check the second pocket: no key. You check the third pocket: yes, the key is there. That is a search because you are looking for one special thing inside a small group of possible locations.

In programming terms, the key is the search target. The pockets are the search space. Each pocket check is a comparison step. The comparison asks, "Is this the thing I want?" If the answer is no, the search continues. If the answer is yes, the search can stop.

This pattern matters because it is the foundation for linear search. The computer does not magically know where the item is unless the data structure gives it a direct path. Without that direct path, it has to inspect candidates one by one. That can be perfectly fine for tiny collections, but it becomes more expensive as the number of items grows.

The important habit is tracing the search in plain English before code. If you can say, "check wallet, check phone, check key," then you already understand the loop pattern: look at each item, compare it to the target, and decide whether to continue.

Search Anatomy

Part	Plain-English meaning	Example
Search target	The specific thing we want to find.	The key
Search space	The group of places or values we are searching through.	Wallet, phone, key
Comparison step	One check asking whether the current item is the target.	Is this item the key?
Stop condition	The reason the search ends.	Found the key or checked everything

Real English Trace

- Check wallet - not the key.
- Check phone - not the key.
- Check key - yes, found it.
- This trace is the human-readable version of the loop the program will later run.

Code Example

```
items = ["wallet", "phone", "key"]
target = "key"

for item in items:
    print("Checking", item)
    if item == target:
        print("Found it")
        break
```

items is the search space, target is the value we want, and the loop visits one item at a time. The if statement is the comparison step. break stops the search early once the target is found.

Made with Goodnotes

Lesson 2 - How Searching Works with Simple Examples (continued)

Terms, Definitions, and Why They Are Called That

- Search space: the set of items or places being checked. It is called a space because it describes the area where the answer could exist.
- Target: the value being searched for. It is the thing the algorithm is trying to hit.
- Candidate: the current item being inspected. It is a candidate because it might be the target.
- Comparison step: the moment the program asks whether the candidate matches the target.
- Break: a command that exits the loop early. It is useful when continuing would waste work after the target is found.

Common Mistakes and Helpful Tips

- Mistake: treating the list itself as the target. The list is where we search; the target is what we want.
- Mistake: forgetting that the loop checks one item per cycle.
- Mistake: continuing to search after the item has already been found when the task only needs the first match.
- Tip: label your variables out loud: search space, target, current item.

Review Checkpoint

- What are the three basic parts of a search?
- What does the comparison step ask?
- Why is break useful after a target is found?
- How can you trace a search in plain English?

Key Takeaways

- A search problem needs a target, a search space, and comparison steps.
- The current item is the candidate being checked right now.
- A loop is a natural way to repeat comparison steps.
- break is useful when the target is found and no more checking is needed.

Notes

Made with Goodnotes

Lesson 3 - Linear Search with Small and Unsorted Data

Big Idea

Linear search checks items one at a time from the beginning of a collection until it finds the target or reaches the end. It is simple, reliable, and especially useful when the data is small or not sorted.

Full Teaching Explanation

Linear search is the most straightforward search algorithm. Imagine looking for a name tag in a messy box. You do not have a sorted order, an index, or a shortcut. You pick up one tag, ask whether it is the one you want, then move to the next. That is linear search.

The word linear means "in a line." The search moves through the data as if the items are lined up from first to last. It does not skip around. It does not split the list in half. It simply checks each item in order. Because of that, it works even when the data is unsorted.

That reliability is the reason linear search is often the first search algorithm taught. It is not always the fastest, but it is easy to understand and hard to misuse. If the target is near the beginning, linear search may finish quickly. If the target is at the end or missing, it may have to check every item.

The key trade-off is simplicity versus speed. For five items, linear search is usually fine. For five million items, checking one by one can become painful. This lesson is training you to recognize when simple is good enough and when a better strategy may be worth the setup.

When Linear Search Makes Sense

Situation	Is linear search reasonable?	Why
Small unsorted list	Yes	There is not enough data to justify a more complex setup.
Short grocery list	Yes	Checking one item at a time is simple and clear.
Huge sorted database	Usually no	Binary search or indexing may be much faster.
Unknown order	Yes	Linear search does not require sorting first.
Repeated searches on same data	Maybe not	A different structure may save work over time.

Mental Model

Picture a row of sticky notes. You start at the left, read one note, compare it to the target, and move right if it is not a match. The search continues in a straight line. That is why the pattern is called linear.

Code Example

```
def linear_search(items, target):
    for index in range(len(items)):
        if items[index] == target:
            return index
    return None

result1 = linear_search([1, 5, 7], 5)
result2 = linear_search([1, 5, 7], 8)
print(result1)
print(result2)
```

The function receives a list and a target. Each loop cycle checks one position. If the value equals the target, the function returns the index immediately. If the loop finishes without finding the target, it returns None.

Code Example

Python

Run Submit code

```
def linear_search(items, target):
    for index in range(len(items)):
        if items[index] == target:
            return index
    return None

result1 = linear_search([3, 7, 9], 7)
print(result1)

result2 = linear_search([4, 5, 8], 2)
print(result2)

result3 = linear_search([1, 2, 3, 4], 4)
print(result3)
```

Output

```
1
None
3
```

Made with Goodnotes

Lesson 3 - Linear Search with Small and Unsorted Data (continued)

Step-by-Step Trace

Check	Current value	Question asked	Result
1	1	Is 1 the target 5?	No - keep going
2	5	Is 5 the target 5?	Yes - return index 1
If searching for 8	1, then 5, then 7	Did any value match 8?	No - return None

Terms, Definitions, and Why They Are Called That

- Linear search: a search that checks items in a straight one-by-one path. It is called linear because the work follows the collection like a line.
- Unsorted data: data that is not arranged in a guaranteed order. Linear search works here because it does not depend on order.
- Return value: the result a function sends back. In this example, the return value is either an index or None.
- None: Python's way of representing no value here. It is useful when the target was not found.

Common Mistakes and Helpful Tips

- Mistake: assuming linear search only works on sorted data. It works on unsorted data because it checks every possible item if needed.
- Mistake: confusing the target with the index. The target is what you want; the index is where it was found.
- Mistake: forgetting the missing-target case. A good function should say what happens when no match exists.
- Tip: when tracing, write the current index, current value, comparison question, and result.
- Tip: linear search is often the right first answer when the data is small, unsorted, or only searched once.

Review Checkpoint

- What are the search target and search space in a search problem?
- What does a comparison step ask?
- Why does linear search work on unsorted data?
- What is the worst-case situation for linear search?
- Why might linear search be fine for a small list but frustrating for a huge list?
- What should a function return if the target is not found?

Key Takeaways

- Searching means locating a target inside a search space.
- Comparison steps are the tiny yes/no checks that power a search.
- Linear search checks items one at a time in order.
- Linear search is simple and reliable for small or unsorted collections.
- The trade-off is that it may check every item when the target is last or missing.

Notes

Made with Goodnotes

Lesson 4 - Binary Search with Sorted Collections

Big Idea

Binary search is a smarter search strategy for sorted collections. Instead of checking every item from the beginning, binary search checks the middle item, decides whether the target must be on the left or right side, and throws away the half that cannot contain the answer.

The power is not magic; it comes from one condition: the data must already be sorted. If the list is not sorted, the left-side-smaller and right-side-bigger logic breaks.

Full Teaching Explanation

Linear search works like looking through flash cards one by one. Binary search works like guessing a number between 1 and 100 when someone tells you too high or too low. Every guess gives information that removes a large chunk of possible answers.

The middle value is the decision point. If the middle value is the target, the search is done. If the target is smaller than the middle value, every value on the right can be ignored because sorted order tells us those values are even larger. If the target is bigger than the middle value, every value on the left can be ignored because those values are too small.

That is why binary search feels so efficient: it does not move one step at a time. It cuts the remaining search space roughly in half each round. With 64 sorted items, linear search might need 64 checks in the worst case. Binary search needs about 6 checks because 64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1.

The beginner trap is using binary search on unsorted data. If the list is scrambled, the middle value does not tell you which half can be ignored. In that situation, linear search is safer because it makes no order assumptions.

Binary Search Mental Model

Step	What binary search does	Why it helps
1. Set low and high	low starts at the first index; high starts at the last index.	These boundaries define the part of the list that is still possible.
2. Find the middle	mid = (low + high) // 2.	The middle gives one comparison that can remove half the search space.
3. Compare middle to target	If values[mid] equals target, the target is found.	The search can stop immediately.
4. Move left or right	If the target is smaller, move high left. If the target is bigger, move low right.	The impossible half is discarded.
5. Stop when empty	If low becomes greater than high, there is no valid search space left.	The target is not in the list.

Think of the search space as a shrinking hallway. The algorithm keeps closing doors until only the target is left or no rooms remain.

Code Example - Binary Search Loop

```
numbers = [1, 3, 5, 7, 9, 11, 13, 15]
target = 7
low = 0
high = len(numbers) - 1

while low <= high:
    mid = (low + high) // 2
    guess = numbers[mid]

    if guess == target:
        print("Found at index", mid)
        break
    elif guess < target:
        low = mid + 1
    else:
        high = mid - 1
```

Plain English: start with the whole sorted list, check the middle, and keep only the half where the target could still be hiding. The list itself does not change; the possible search area changes.

Made with Goodnotes

Trace Practice - Watch the Boundaries Move

Round	low	high	mid	guess	Decision
1	0	7	3	7	Target found immediately.
If target were 13	0	7	3	7	13 is larger, so move low to 4.
Next round	4	7	5	11	13 is larger, so move low to 6.
Next round	6	7	6	13	Target found.

When tracing binary search, track low, high, mid, and guess. Those four values explain the entire search.

Terms, Definitions, and Why They Are Called That

- Binary search: a search method that repeatedly divides remaining possibilities into two parts. It is called binary because each comparison chooses between two directions: left half or right half.
- Sorted collection: data arranged in order, such as smallest to largest or A to Z. Binary search needs this order to decide which half is impossible.
- Low: the lowest index still being considered. It starts at the beginning and moves right when the target must be larger.
- High: the highest index still being considered. It starts at the end and moves left when the target must be smaller.
- Mid: the middle index between low and high. It is the spot binary search checks first each round.
- Search space: the part of the data where the target could still be. In binary search, the search space shrinks fast.

Binary Search Practice Lab

Practice prompt	How to think it through	Answer pattern
Sorted list has 64 items. What is the maximum number of checks?	Keep halving: 64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1.	About 6 checks.
The list is [12, 3, 18, 7, 1]. Should you use binary search?	Ask the sorted question first. This list is scrambled.	No. Use linear search unless the list is sorted first.
Target is smaller than the middle value.	Sorted order tells us everything to the right is even bigger.	Move high left. Search the left half.
Target is bigger than the middle value.	Sorted order tells us everything to the left is too small.	Move low right. Search the right half.

Common Mistakes and Helpful Tips

- Mistake: using binary search on unsorted data. If the list is scrambled, the middle value cannot tell you which side to ignore.
- Mistake: moving low or high the wrong direction. If guess is too small, move low right. If guess is too large, move high left.
- Mistake: forgetting that indexes start at 0. The middle value and middle index are related but not the same thing.
- Tip: write low, high, mid, and guess in a small table when solving quiz questions.
- Memory hook: binary search does not search harder; it searches smarter by deleting half the possibilities each time.

Out Loud Programmer Thinking

I am looking for a target inside an ordered list. I do not need to check every value because sorted order lets me eliminate values that cannot possibly work.

First I check the middle. If the middle is too small, I move right. If the middle is too large, I move left. If the boundaries cross, the target is not there.

Before I choose binary search, I must ask: is this data sorted? If not, the smart-looking algorithm becomes a confidently wrong algorithm, which is rude but very on brand for bugs.

Notes

Made with Goodnotes

Lesson 5 - Comparing Linear and Binary Search

Big Idea

Linear search and binary search both try to find a target, but they make different promises. Linear search promises flexibility: it works even when the data is small, random, or unsorted. Binary search promises speed: it can remove half the remaining choices each step, but only when the data is sorted.

The real skill is not memorizing which one is better. The real skill is choosing the right search method for the situation.

Full Teaching Explanation

Linear search is the safe beginner pattern because it asks the simplest possible question over and over: is this the target? It starts at the beginning and keeps checking until it finds the answer or runs out of data. This is easy to read, easy to test, and works on almost any list.

Binary search is faster when the collection is sorted because it uses order as information. If the target is smaller than the middle value, the right half cannot contain the target. If the target is larger, the left half cannot contain the target. That lets the algorithm skip huge chunks of work.

The trade-off is that binary search has a prerequisite. The data must already be sorted in the same order the algorithm expects. If sorting has already been done or the data naturally stays sorted, binary search is excellent. If the list is tiny, constantly changing, or not sorted, linear search may be simpler and perfectly reasonable.

Linear vs Binary Search Decision Table

Question	Linear Search	Binary Search
Does the data need to be sorted?	No. It can handle random or unsorted order.	Yes. Sorting/order is required.
How does it move?	Checks one item at a time from start to finish.	Checks the middle and cuts the search space in half.
Best for	Small lists, unsorted lists, quick simple checks.	Large sorted lists or repeated searches on ordered data.
Worst-case work	May check every item.	Checks only about log2(n) positions.
Beginner risk	Mostly just forgetting to stop or return.	Using it on unsorted data or moving boundaries wrong.

Step Count Comparison

List size	Linear search worst case	Binary search worst case	What that means
5 items	5 checks	about 3 checks	Both are fine; simplicity may matter more.
10 items	10 checks	about 4 checks	Binary starts showing an advantage.
20 items	20 checks	about 5 checks	Binary is much more efficient if sorted.
64 items	64 checks	about 6 checks	Repeated halving becomes a big win.
1,000 items	1,000 checks	about 10 checks	Binary avoids almost all of the work.

The exact binary count may vary slightly depending on implementation and target position, but the growth pattern is the important idea: linear grows one step at a time, while binary grows by halving.

Choosing the Right Search

Situation	Choose	Why
A tiny unsorted grocery list	Linear search	Simple and reliable; no sorting required.
A dictionary or phonebook sorted A to Z	Binary search	Order lets you jump to the middle and discard half.
A constantly changing list	Probably linear first	Sorting over and over may waste more work than it saves.
A huge sorted database field	Binary search or indexed lookup	Large sorted data rewards faster search strategies.
A quiz says sorted list and faster search	Binary search	Those words are clues that halving is allowed.

Made with Goodnotes

Common Mistakes and Helpful Tips

- Mistake: assuming binary search is always better. It is only better when its requirements are met.
- Mistake: forgetting the first decision question: is the data sorted?
- Mistake: measuring only speed and ignoring readability. A simple linear search can be the better choice for small data.
- Tip: for quiz questions, ask two things first: is it sorted, and is it big?
- Memory hook: linear steps grow one by one; binary steps grow much slower because each step cuts the list in half.

Notes

Made with Goodnotes

Lesson 6 - Challenge: Searching

Big Idea

The challenge lesson asks you to choose a search strategy, trace what the code does, and explain why the choice is correct. The goal is to prove that you understand the rules, not just recognize vocabulary words.

For search questions, the safest process is: identify the target, identify the search space, decide whether the list is sorted, choose linear or binary search, and then trace the comparisons.

Full Teaching Explanation

This lesson acts like a mini test of everything so far. Some prompts ask which search strategy fits. Others ask what a piece of code prints. Others ask whether a statement about binary search is true. They all come back to the same foundation: searching means looking for a target inside a search space.

If the data is not sorted, linear search is usually the correct beginner answer. Linear search does not care whether the numbers are in order. It checks each item until the target is found or the list ends. This makes it reliable for messy, short, or unpredictable data.

If the data is sorted, binary search becomes possible. Binary search is usually chosen when the list is sorted and the question hints at speed, large data, or fewer checks. But the sorted condition is not decorative; it is the entire reason binary search can throw away half the values.

The code-tracing part trains careful reading. A variable like found usually starts as False. If the loop sees the target, the code changes found to True. When the loop ends, print(found) tells you whether the target appeared at least once.

Challenge Strategy Table

Prompt clue	Best thinking move	Likely answer
The list is not sorted	Do not assume halves mean anything.	Linear search
The list is sorted	Check whether the question wants faster/fewer checks.	Binary search may fit
The target might not exist	Trace until found or until search space is empty.	Return not found / False
The code uses found = False	Watch whether an if statement changes it to True.	Final print depends on the condition
The question asks for checks	Count comparisons, not just lines of code.	Use trace steps

Code Trace - Found Flag

```
nums = [2, 5, 8, 12, 25, 30]
target = 25
found = False

for n in nums:
    if n == target:
        found = True

print(found)
```

Plain English: start by assuming the target has not been found. Check each number. If any number equals the target, switch found to True. Because 25 appears in the list, the final output is True.

Code Choice - Why Linear Search Fits Unsorted Data

```
nums = [12, 5, 8, 2, 30, 25]
target = 25
# This list is not sorted.
# Binary search should NOT be used here.

for n in nums:
    if n == target:
        print("Found")
        break
```

Plain English: because the list is scrambled, the program should check one value at a time. Without sorted order, the middle value does not safely eliminate a half.

Made with Goodnotes

Search Challenge Checklist

Question to ask	Why it matters
What is the target?	You cannot search if you do not know what value you want.
What is the search space?	This tells you where the algorithm is allowed to look.
Is the data sorted?	This decides whether binary search is even allowed.
How many comparisons happen?	This is the actual work the search performs.
What stops the search?	The algorithm must know when it found the target or when no possibilities remain.

Lesson 6 Quick Recap

- Linear search is the safe choice for small or unsorted collections.
- Binary search is the fast choice for sorted collections because each check can remove half the remaining search space.
- Challenge questions often test whether you notice the sorted requirement.
- Code-tracing questions require following variable changes one step at a time.
- A strong answer explains why the strategy fits, not just which strategy was chosen.

Notes

Made with Goodnotes

Lesson 7 - Why Sorting Is Useful

Big Idea

Sorting means arranging data into a useful order. It can be alphabetical, numerical, chronological, by client name, by date, by score, or by any other rule that helps a person or program use the information more easily.

Sorting is useful because organized data is easier to search, easier to scan, easier to compare, and easier to report. It is not only about making a list look neat. It is about reducing confusion and making later work simpler.

Full Teaching Explanation

This lesson begins with everyday examples because sorting is something you already rely on constantly. Contacts are easier to find when names are alphabetized. Books are easier to locate when arranged by title, author, or subject. Photos from a shoot are easier to deliver when grouped by client, session, date, or rating instead of dumped into one chaotic folder with filenames that look like they were named by a sleep-deprived robot.

In programming, sorting does the same kind of organizing. A list like ["Zoe", "Ava", "Mason", "Ben"] contains the right names, but it is harder to scan than ["Ava", "Ben", "Mason", "Zoe"]. The values did not change. The order changed. That order can make humans faster, and it can make some algorithms possible.

The connection to searching is important. Linear search can work on unsorted data, but it may need to check many items. Binary search only works correctly when the list is sorted. That means sorting can be a setup step: you spend some work up front to organize the data so later searching, reading, or reporting becomes easier.

Sorting is not automatically free. It takes work. If the list is tiny or changes constantly, sorting again and again may not be worth it. If the list is large, reused often, or needs to be presented clearly, sorting can pay off.

Real-World Sorting Examples

Situation	Useful sort order	Why sorting helps
Phone contacts	Alphabetical by name	You can jump near the person instead of scrolling randomly.
Horse show photos	By rider, class, date, or session	Finding deliverables and client galleries becomes faster and less chaotic.
Grades or scores	Lowest to highest or highest to lowest	It reveals rank, top scores, outliers, and patterns.
File folders	By date or project	Related work stays together and repeated searching takes less effort.
Numbers before binary search	Smallest to largest	The sorted order makes halving possible.

Code Example - Sorting Names

```
names = ["Zoe", "Ava", "Mason", "Ben"]
sorted_names = sorted(names)

print(names)
print(sorted_names)
```

Plain English: keep the original list, create a sorted version, and print both so you can see that sorted() returns a new ordered list. The original names list is still in its original order.

Code Example - Sorting Records by One Field

```
sessions = [
    {"client": "Avery", "date": "2026-05-01"},
    {"client": "Morgan", "date": "2026-04-18"},
    {"client": "Avery", "date": "2026-04-21"}
]

by_client = sorted(sessions, key=lambda s: s["client"])
print(by_client)
```

Plain English: each session is a small record. The key tells Python which part of each record to compare. Sorting by client groups related sessions together before you search through dates or files.

Made with Goodnotes

Sorting Decision Questions

Question	What it tells you
Will a human need to scan this list?	Sorting may improve readability and reduce frustration.
Will the program search this data many times?	Sorting once may make later search faster.
Does binary search need to be used?	The list must be sorted first.
Does the data change constantly?	Repeated sorting may waste work.
Is there a meaningful order?	Sorting needs a rule: alphabetic, numeric, date, rank, etc.

Terms, Definitions, and Why They Are Called That

- Sorting: arranging values into a chosen order. It is called sorting because items are grouped or ordered by a rule instead of left random.
- Sort key: the value used to decide order, such as name, date, score, or client. It is called a key because it unlocks what part of each item should be compared.
- Ascending order: smallest-to-largest or A-to-Z order. Ascending means moving upward.
- Descending order: largest-to-smallest or Z-to-A order. Descending means moving downward.
- Setup cost: work done early so later work becomes easier. Sorting can be a setup cost before faster searches or cleaner reports.

Common Mistakes and Helpful Tips

- Mistake: thinking sorting changes the data itself. Usually, the values stay the same; only their order changes.
- Mistake: sorting without knowing the goal. Alphabetical order helps names, but date order may help sessions, and score order may help rankings.
- Mistake: assuming sorting is always worth it. If the data is tiny or constantly changing, sorting may not save enough work.
- Tip: ask what order would make the next task easier: search, scan, compare, display, or report.
- Memory hook: sorting is not decoration; it is organization that makes later work easier.

Notes

Made with Goodnotes

Lesson 8 - Sorting Different Approaches

Big Idea

There is more than one way to sort. Different sorting approaches follow different strategies, even when they produce the same final ordered list.

This lesson introduces sorting as a family of algorithms. Some approaches repeatedly compare neighbors. Some repeatedly select the smallest remaining item. Some insert each new item into the right place. The final goal is the same, but the thinking pattern differs.

Full Teaching Explanation

Imagine several people trying to organize numbered cards on a table. One person might compare neighboring cards and swap them whenever the left card is bigger than the right. Another person might scan the whole row, find the smallest card, and place it at the front. Another person might pick up cards one at a time and insert each card into the correct place among the cards already sorted.

Those are all sorting strategies. Bubble sort is the neighbor-swapper. Selection sort is the smallest-card hunter. Insertion sort is the careful card-inserter. Each one gives the computer a rule for turning disorder into order.

Python gives us sorted() and .sort(), but those are convenient tools built on algorithmic ideas. Learning simple sorting approaches helps you understand what it means for a program to compare values, move values, repeat passes, and stop when order is achieved.

The big beginner skill is recognizing the story each algorithm tells. Bubble-style thinking asks: are these two neighbors in the wrong order? Selection-style thinking asks: what is the smallest remaining item? Insertion-style thinking asks: where does this new item belong in the part I have already organized?

Sorting Approaches Compared

Approach	Core question	Mental model
Bubble-style thinking	Are these neighboring items in the wrong order?	Walk across the list swapping neighbors until large values drift right.
Selection-style thinking	What is the smallest remaining item?	Find the smallest card and place it into the next correct spot.
Insertion-style thinking	Where does this item fit among the sorted items?	Build a sorted hand of cards one item at a time.
Python built-in sort	Can the language sort this for me safely?	Use sorted() or .sort() when the goal is sorting, not practicing the algorithm.

Neighbor-Swapping Strategy

Step	Neighbor-swap / bubble-style thinking
1	Look at two neighboring values.
2	If the left value is bigger than the right value, swap them.
3	Move one position to the right and check the next pair.
4	After one full pass, the largest value has moved toward the end.
5	Repeat passes until no more swaps are needed or the list is ordered.

Find-the-Smallest Strategy

Step	Selection-style thinking
1	Look through the unsorted part of the list.
2	Find the smallest remaining value.
3	Swap or place that value into the next correct position.
4	Treat that position as finished.
5	Repeat for the remaining unsorted values.

This is the mindset behind selection sort. It is not swapping every messy neighbor; it is choosing the next best item and locking it into place.

Made with Goodnotes

Code Example - Built-In Sort vs Algorithm Thinking

```
numbers = [7, 1, 4, 2]

# Built-in sorting for real projects
ordered = sorted(numbers)
print(ordered)

# Algorithm practice asks a different question:
# "How could we build the sorting steps ourselves?"
```

Plain English: in normal Python work, sorted() is usually the clean choice. But this lesson is about the thinking underneath sorting: repeated comparisons and movements.

How to Tell Sorting Approaches Apart

Question	Bubble-style answer	Selection-style answer	Insertion-style answer
What does it focus on?	Neighbor pairs	Smallest remaining item	Correct position for the current item
What movement does it use?	Repeated swaps	Choose and place/swap	Shift values and insert
Why beginners like it	Very visual and easy to trace	Clear goal each round	Feels like sorting cards by hand
Main warning	Can be slow for large lists	Still scans repeatedly	Shifting can be confusing at first

Terms, Definitions, and Why They Are Called That

- Swap: exchanging two values. It is called a swap because each value takes the other value's place.
- Pass: one trip through part of the list. It is called a pass because the algorithm passes across the data checking or moving values.
- Neighbor comparison: comparing two values next to each other. Bubble sort relies on neighbor comparisons.
- Selection: choosing one item from a group. Selection sort selects the smallest remaining item each round.
- Insertion: placing an item into the right spot. Insertion sort inserts each item into the sorted portion.

Mini Practice - Match the Sorting Story

Clue in the question	Approach it hints at	Why
Compare neighboring items and swap if wrong	Bubble-style thinking	The algorithm only looks at adjacent pairs.
Find the smallest remaining card	Selection-style thinking	Each round selects one value and locks it into place.
Insert each new value into an already sorted hand	Insertion-style thinking	The sorted portion grows one item at a time.
Use Python to get a clean ordered copy	sorted()	The goal is the final order, not practicing the manual algorithm.

When a quiz describes the behavior, translate it into the story. Neighbor swaps point to bubble sort. Smallest remaining value points to selection sort. A growing sorted hand points to insertion sort.

Common Mistakes and Helpful Tips

- Mistake: thinking all sorting algorithms work the same way. They may produce the same result while using very different steps.
- Mistake: mixing up bubble and selection. Bubble swaps neighbors; selection hunts for the smallest remaining item.
- Mistake: focusing only on the final sorted list. Sorting questions usually care about the process, not only the answer.
- Tip: describe the algorithm as a story: swap neighbors, find the smallest, or insert into place.
- Memory hook: same destination, different roads.

Notes

Made with Goodnotes

Lesson 9 - Bubble Sort

Big Idea

Bubble sort is a sorting algorithm that repeatedly compares neighboring items and swaps them when they are in the wrong order. After each pass, larger values drift toward the end of the list, like bubbles rising through water.

Bubble sort is not famous because it is the fastest. It is useful for learning because its logic is easy to see: compare neighbors, swap if needed, keep repeating until the list is sorted.

Full Teaching Explanation

Bubble sort starts with a simple local question: are these two neighboring values in the correct order? If the left value is bigger than the right value, they are out of order for ascending sorting, so the algorithm swaps them. Then it moves to the next pair and asks the same question again.

One pass does not necessarily sort the whole list. One pass moves the largest unsorted value toward the right side. For example, in [5, 1, 4, 2, 8], the 5 swaps past 1, then past 4, then past 2. Large values gradually drift toward their final positions.

The word bubble comes from that movement. Large values gradually move to the end through repeated swaps, similar to bubbles rising upward. The algorithm does not know the final position immediately. It discovers order by comparing adjacent pairs over and over.

Bubble sort is easy to trace, but it can do a lot of repeated work. With nested loops, it often behaves like O(n^2): as the list gets longer, the number of pair comparisons grows quickly. That is why bubble sort is mostly a learning tool, not the sort you would normally choose for a large production dataset.

Bubble Sort Step Pattern

Step	What happens	Plain-English meaning
1	Start at the beginning of the list.	Look at the first pair of neighbors.
2	Compare the left and right neighbor.	Ask whether they are in the wrong order.
3	Swap if the left value is larger.	Move the bigger value to the right.
4	Move to the next pair.	Keep checking neighbors across the list.
5	Repeat passes.	Keep bubbling large values toward the end until sorted.

Trace Example - One Bubble Sort Pass

Comparison	List after comparison	Why
5 and 1	[1, 5, 4, 2, 8]	5 is bigger than 1, so swap.
5 and 4	[1, 4, 5, 2, 8]	5 is bigger than 4, so swap.
5 and 2	[1, 4, 2, 5, 8]	5 is bigger than 2, so swap.
5 and 8	[1, 4, 2, 5, 8]	5 is smaller than 8, so leave them.
End of pass	[1, 4, 2, 5, 8]	A large value has moved toward the right side.

Code Example - One Pass of Bubble Sort

```
numbers = [5, 1, 4, 2, 8]

for i in range(len(numbers) - 1):
    if numbers[i] > numbers[i + 1]:
        temp = numbers[i]
        numbers[i] = numbers[i + 1]
        numbers[i + 1] = temp

print("After one pass:", numbers)
```

Plain English: walk across neighboring pairs. If the left number is bigger, swap it with the number on the right. This is one pass, not the full sort.

Made with Goodnotes

Code Example - Repeated Passes

```
numbers = [8, 7, 1, 3, 6]

for pass_num in range(len(numbers) - 1):
    for i in range(len(numbers) - 1):
        if numbers[i] > numbers[i + 1]:
            temp = numbers[i]
            numbers[i] = numbers[i + 1]
            numbers[i + 1] = temp

print(numbers)
```

Plain English: the outer loop repeats passes. The inner loop compares neighboring values during each pass. This nested-loop shape is why bubble sort is commonly described as O(n^2) in beginner Big O terms.

Bubble Sort Concepts and Efficiency

Idea	Meaning for bubble sort
Neighbor comparison	Only two adjacent values are checked at a time.
Swap	Values trade places when they are out of order.
One pass	A walk across the list that pushes a large value toward the end.
Repeated passes	The process continues because one pass usually does not finish the whole sort.
O(n^2)	Nested repeated comparisons can grow quickly as list size increases.

Terms, Definitions, and Why They Are Called That

- Bubble sort: a sorting algorithm where large values bubble toward the end through repeated neighbor swaps.
- Adjacent values: values next to each other. Bubble sort only compares adjacent values at each moment.
- Swap: exchange two values so they trade positions.
- Pass: one complete walk through the list comparing neighbor pairs.
- Nested loop: a loop inside another loop. Bubble sort often uses nested loops to repeat passes and comparisons.
- O(n^2): a growth pattern where work increases roughly with n times n.

Common Mistakes and Helpful Tips

- Mistake: thinking one pass always sorts the entire list. One pass often only moves a large value closer to the end.
- Mistake: comparing every item to every other item randomly. Bubble sort compares neighbors in order.
- Mistake: forgetting the temporary variable during a manual swap.
- Mistake: calling bubble sort fast just because it is easy to understand. It is usually not efficient for large lists.
- Tip: trace bubble sort with arrows between neighbor pairs. Write the list after each swap.
- Memory hook: bubble sort = compare neighbors, swap if needed, repeat until sorted.

Out Loud Programmer Thinking

I am not trying to find the smallest item directly. I am walking across the list and cleaning up neighbor pairs. If the left neighbor is bigger than the right neighbor, I swap them. If not, I leave them. After enough passes, the list becomes ordered. This is wonderfully visual and wonderfully inefficient - the algorithm equivalent of tidying a room by picking up one sock at a time.

Notes

Made with Goodnotes

Code Example

Python

Run

Submit code

```python
numbers = [8, 7, 1, 3, 6]

for i in range(len(numbers) - 1):      # one pass
    print("Comparing index", i, "and", i + 1, "→", numbers)
    if numbers[i] > numbers[i + 1]:    # if left is bigger
        # swap them
        temp = numbers[i]
        numbers[i] = numbers[i + 1]
        numbers[i + 1] = temp
        print("  swapped →", numbers)

print("After one pass:", numbers)
```

Output

```
Comparing index 0 and 1 → [8, 7, 1, 3, 6]
  swapped → [7, 8, 1, 3, 6]
Comparing index 1 and 2 → [7, 8, 1, 3, 6]
  swapped → [7, 1, 8, 3, 6]
Comparing index 2 and 3 → [7, 1, 8, 3, 6]
  swapped → [7, 1, 3, 8, 6]
Comparing index 3 and 4 → [7, 1, 3, 8, 6]
  swapped → [7, 1, 3, 6, 8]
After one pass: [7, 1, 3, 6, 8]
```

Made with Goodnotes

Lesson 10 - Insertion Sort

Big Idea

Insertion sort builds a sorted section one item at a time. It looks at the next unsorted value, called the key, and inserts that value into the correct position inside the already-sorted left side of the list. The mental model is sorting cards in your hand: each new card gets compared against the cards you already placed, then slides into the right spot.

Full Teaching Explanation

Insertion sort is one of the friendliest sorting algorithms because it matches something people already do without thinking. If you pick up playing cards one at a time, you do not usually throw the whole hand into chaos every time a new card appears. You keep the left side sorted and place each new card where it belongs.

The algorithm divides the list into two regions. The left region is treated as sorted. The right region is still unsorted. At the beginning, the first item by itself is considered sorted because one item is automatically in order. Then the algorithm takes the next value, stores it as the key, and compares it backward through the sorted region.

If the previous value is larger than the key, that previous value shifts one spot to the right. This creates space. The algorithm keeps shifting until it either reaches the beginning of the list or finds a value that is smaller than or equal to the key. Then the key is inserted into the empty spot. That is why the algorithm is called insertion sort: the main action is inserting each next value into the correct place.

Insertion sort is not the fastest general-purpose algorithm for huge random data. In the worst case, it can still require many comparisons and shifts. But it is excellent for learning because it teaches sorted-region thinking, backward comparison, shifting values, and the difference between swapping and inserting.

Card-Hand Mental Model

Part of the analogy	Sorting meaning	Why it matters
Cards already in your hand	The sorted left side	This portion is already in correct order.
New card you pick up	The key value	This is the value being inserted.
Sliding bigger cards right	Shifting larger elements	This makes room for the key.
Putting the card in place	Insertion step	The key lands where the order stays correct.

Code Example - Insertion Sort in Python

```
numbers = [8, 7, 1, 3, 6]

for i in range(1, len(numbers)):
    key = numbers[i]
    j = i - 1

    while j >= 0 and numbers[j] > key:
        numbers[j + 1] = numbers[j]
        j -= 1

    numbers[j + 1] = key

print(numbers)
```

Plain English: start with the second item. Treat everything to its left as sorted. Move left while values are bigger than the key, shift those bigger values right, then place the key into the open spot.

Line-by-Line Explanation

- **for i in range(1, len(numbers))**: starts at index 1 because index 0 is already a sorted section of one item.
- **key = numbers[i]**: saves the value we are trying to insert. This matters because shifting values will overwrite positions.
- **j = i - 1**: begins comparison immediately to the left of the key, inside the sorted section.
- **while j >= 0 and numbers[j] > key**: keeps moving left only while there is still a valid index and the value is too large to stay before the key.
- **numbers[j + 1] = numbers[j]**: shifts the larger value one space to the right. This is not a swap; it is making room.
- **j -= 1**: moves the comparison one step farther left.
- **numbers[j + 1] = key**: places the saved key in the open location after the shifting stops.

Made with Goodnotes

Trace Example - First Few Insertions

Pass	Key	What gets shifted?	List after insertion
Start	-	-	[8, 7, 1, 3, 6]
i = 1	7	8 shifts right	[7, 8, 1, 3, 6]
i = 2	1	8 shifts, then 7 shifts	[1, 7, 8, 3, 6]
i = 3	3	8 shifts, then 7 shifts	[1, 3, 7, 8, 6]
i = 4	6	8 shifts, then 7 shifts	[1, 3, 6, 7, 8]

Terms, Definitions, and Why They Are Called That

- Insertion sort: a sorting algorithm that inserts each next value into its correct place within an already sorted section. It is called insertion sort because the key action is inserting, not repeatedly swapping every neighbor pair.
- Key: the value currently being placed. It is called the key because it is the value we hold onto while we decide where it belongs.
- Sorted section: the part of the list that has already been organized. In insertion sort, this usually grows from left to right.
- Shift: moving a value one position to the right. This creates the opening where the key can be inserted.
- In-place sort: a sort that rearranges the original list instead of building a completely separate sorted list.

Common Mistakes and Helpful Tips

- Mistake: calling every movement a swap. In insertion sort, larger values usually shift right while the key waits.
- Mistake: forgetting to save the key before shifting. If you do not save it, you can overwrite the value you meant to insert.
- Mistake: starting the loop at 0. The algorithm usually starts at index 1 because the first item alone is already sorted.
- Mistake: inserting at j instead of j + 1. The loop stops one step before the correct opening.
- Tip: say it out loud: hold the key, shift bigger values right, drop the key into the opening.

Notes

Code Example

Python ▷ Run ↙ Submit code

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -=1
        arr[j + 1] = key

    return arr

print(insertion_sort([1, 2, 4, 3, 5]))
```

Output

```
[1, 2, 3, 4, 5]
```

Made with Goodnotes

Lesson 11 - Bubble Sort vs Insertion Sort

Big Idea

Bubble sort and insertion sort are both beginner-friendly sorting algorithms, and both can be slow on large random lists. The difference is how they move values. Bubble sort repeatedly compares neighbors and swaps them. Insertion sort builds a sorted left side and inserts each new value where it belongs.

Full Teaching Explanation

This lesson matters because two algorithms can have the same broad goal - sorting a list - while using completely different strategies. Bubble sort is neighbor-focused. It walks across the list asking, "Are these two neighbors in the wrong order?" If yes, it swaps them. After one full pass, a large value tends to bubble toward the end.

Insertion sort is placement-focused. It asks, "Where does this next value belong inside the part I already sorted?" Instead of swapping adjacent pairs again and again, it holds a key value and shifts larger values to make space. Once the opening is found, the key is inserted.

Both can take O(n^2) time in the worst case because they may require repeated comparisons as the list grows. But insertion sort often feels smarter on small or nearly sorted data because it can do very little work when values are already close to where they belong. Bubble sort is easier to visualize but often less efficient in practice.

The point is not to memorize one as "good" and one as "bad." The point is to recognize the shape of the strategy. Bubble sort is repeated neighbor cleanup. Insertion sort is sorted-hand building. Once you can see those shapes, code tracing becomes much less chaotic.

Side-by-Side Decision Table

Question	Bubble sort answer	Insertion sort answer
Main action	Compare neighboring values and swap if needed.	Insert the key into the sorted left side.
Mental model	Bubbles rising to the end of the list.	Sorting cards in your hand.
Movement style	Many neighbor swaps.	Shifts larger values right, then places key.
Best beginner clue	Nested loops with adjacent comparison like numbers[i] > numbers[i + 1].	A key variable, j moving backward, and shifting values.
Efficiency idea	Usually O(n^2), simple but not great for large data.	Also O(n^2) worst case, but often better for nearly sorted data.

Code Shape Comparison

```
# Bubble sort clue: neighbor comparison and swap
if numbers[j] > numbers[j + 1]:
    temp = numbers[j]
    numbers[j] = numbers[j + 1]
    numbers[j + 1] = temp

# Insertion sort clue: save key, shift larger values right
key = numbers[i]
while j >= 0 and numbers[j] > key:
    numbers[j + 1] = numbers[j]
    j -= 1
numbers[j + 1] = key
```

Plain English: bubble sort keeps trading neighboring values. Insertion sort holds one value aside, slides bigger values over, and inserts the held value into the open spot.

How to Tell Which Algorithm You Are Looking At

Code clue	Usually means	Why
numbers[i] > numbers[i + 1]	Bubble sort	The algorithm is comparing immediate neighbors.
temp = numbers[i]	Bubble sort swap	A temporary variable often helps trade two values.
key = numbers[i]	Insertion sort	The algorithm saves the value it wants to place.
j -= 1 inside a while loop	Insertion sort	The comparison walks backward through the sorted section.
numbers[j + 1] = numbers[j]	Insertion sort shift	A larger value slides right to make room for the key.

Made with Goodnotes

Trace Thinking: Same List, Different Strategy

For the list [8, 7, 1, 3, 6], bubble sort starts by comparing 8 and 7, swapping them, then comparing 8 and 1, swapping again, and so on. The large value 8 gets pushed to the right through repeated neighbor swaps.

Insertion sort starts with 7 as the key. It compares 7 against 8, shifts 8 right, and inserts 7 at the front. Then it uses 1 as the next key, shifts 8 and 7 right, and inserts 1 at the front. Instead of pushing one large value through neighbors, it keeps inserting the next card into a sorted hand.

When a quiz asks what happens after one pass, be careful. "One pass" means different things depending on the algorithm. In bubble sort, one pass usually means one left-to-right neighbor sweep. In insertion sort, one pass usually means placing one key into the sorted section.

Common Mistakes and Helpful Tips

- Mistake: assuming both algorithms do the same work because both sort. Same goal, different movement pattern.
- Mistake: treating insertion sort as bubble sort with different names. Insertion sort shifts and inserts; bubble sort swaps neighbors.
- Mistake: saying O(n^2) means the algorithms are identical. Big O describes growth, not the exact behavior or readability.
- Tip: find the variable names. key usually screams insertion sort. adjacent indexes usually scream bubble sort.
- Memory hook: bubble sort cleans neighbor pairs; insertion sort builds a sorted hand.

Key Takeaways

- Bubble sort moves values through repeated neighbor swaps.
- Insertion sort moves the key into the sorted section by shifting larger values.
- Both can be O(n^2), especially on large random input.
- Insertion sort can be efficient on small or nearly sorted lists.
- Algorithm comparison should include readability, amount of movement, prerequisites, and growth rate.

Notes

Made with Goodnotes

Lesson 12 - Challenge: Sorting

Big Idea

Sorting challenges test whether you can recognize the sorting pattern, trace the list step by step, and explain why the algorithm changes the list the way it does. The goal is not just to get the final sorted list. The goal is to understand the work that happened along the way.

Full Teaching Explanation

Challenge lessons are where the course asks you to stop watching the algorithm and start driving it. You may be given a list and asked what happens after one pass, which algorithm is being used, which values are compared, or whether the list is sorted correctly. The safest strategy is to slow down and identify the pattern before touching the answer choices.

If the challenge shows neighboring values being compared and swapped, think bubble sort. If the challenge shows a key value being held while larger values shift to the right, think insertion sort. If the challenge asks whether a list must be sorted before searching, connect back to binary search: binary search only works when the data is sorted.

The most common trap is jumping straight to the final sorted list when the question only asks for one pass or one step. A full sort and one pass are not the same thing. Another trap is assuming all sorting algorithms move values the same way. Bubble sort, insertion sort, and other approaches may all produce sorted output, but the intermediate steps look different.

A good challenge answer explains the reason, not only the result. "The list becomes [1, 3, 6, 7, 8]" is useful, but "the key moved left after larger values shifted right" shows real understanding. That is the difference between guessing and tracing.

Sorting Challenge Checklist

Question to ask	What it tells you
Is the list supposed to be fully sorted or only changed after one pass?	Prevents doing too much work.
Are neighbors being compared?	Likely bubble sort or another adjacent-swap method.
Is there a key value being held aside?	Likely insertion sort.
Are larger values shifting right?	Insertion sort is making room for the key.
Does the question mention binary search?	Confirm the data is sorted before using it.
What exactly is the output asking for?	Final list, one pass, number of checks, or algorithm name.

Practice Trace - Bubble Sort One Pass

List: [5, 1, 4, 2, 8]

Goal: Show the list after one left-to-right bubble pass.

Comparison	Action	List after action
5 and 1	Swap because 5 > 1	[1, 5, 4, 2, 8]
5 and 4	Swap because 5 > 4	[1, 4, 5, 2, 8]
5 and 2	Swap because 5 > 2	[1, 4, 2, 5, 8]
5 and 8	No swap because 5 < 8	[1, 4, 2, 5, 8]

After one bubble pass, the largest value seen during the pass has moved rightward. The list is not fully sorted yet, and that is the whole point of the challenge trap.

Made with Goodnotes

Practice Trace - Insertion Sort One Step

List before step: [2, 4, 7, 3, 8]

Assume the sorted section is [2, 4, 7] and the next key is 3.

Step	Action	List state / meaning
Save key	key = 3	Hold 3 aside before shifting.
Compare 7 > 3	Shift 7 right	[2, 4, 7, 7, 8] temporarily
Compare 4 > 3	Shift 4 right	[2, 4, 4, 7, 8] temporarily
Compare 2 > 3	False, stop shifting	The key belongs after 2.
Insert key	Place 3 at index 1	[2, 3, 4, 7, 8]

Challenge Question Types

Challenge type	What to do first	Common trap
Identify the algorithm	Look for neighbor swaps vs key/shift behavior.	Only looking at the final sorted result.
Predict one pass	Trace exactly one outer-loop pass.	Accidentally fully sorting the list.
Count comparisons	Count only the comparisons the algorithm actually makes.	Counting every possible pair randomly.
Spot the bug	Check indexes, loop bounds, saved key, and insertion location.	Assuming the syntax-looking part is correct.
Choose search after sorting	Ask whether binary search is now allowed.	Forgetting binary search requires sorted data.

Code Debugging Checklist

- For bubble sort, check that neighboring values are compared: item i with item i + 1.
- For bubble sort, check that the swap does not overwrite one value before saving it in temp.
- For insertion sort, check that the key is saved before shifting begins.
- For insertion sort, check that the while loop moves backward and stops when the correct spot is found.
- For insertion sort, check that the key is inserted at j + 1, not j.
- For any sorting challenge, write the list after each visible change. Your eyes are dramatic and will absolutely lie to you.

Terms, Common Mistakes, and Helpful Tips

- Trace: manually following an algorithm step by step. It is called a trace because you are tracking the path the data takes.
- Pass: one major sweep or cycle through part of the algorithm. The meaning depends on the algorithm, so define it before answering.
- Comparison: checking two values to decide order. Sorting is built out of many small comparisons.
- Swap: trading two values. Bubble sort uses swaps heavily.
- Shift: moving a value to make room. Insertion sort uses shifting heavily.
- Tip: write the question goal at the top of your scratch work: final list, one pass, one step, algorithm name, or number of checks.

Notes

Made with Goodnotes

Lesson 13 - Using Python Sorting Tools in Practice

Big Idea

Python gives you built-in sorting tools so you do not always have to write bubble sort or insertion sort by hand. The big practical tools are list.sort(), sorted(), reverse=True, and key=. These tools let you sort numbers, strings, tuples, and records in a clean, readable way.

This lesson connects the earlier manual sorting algorithms to real Python habits. Bubble sort and insertion sort teach how sorting works underneath. Python sorting tools are what you usually reach for when you need a polished program to arrange data efficiently.

Full Teaching Explanation

The earlier sorting lessons focused on how a list can be rearranged step by step. That matters because it builds the mental model: comparisons decide order, and values move until the collection has a useful arrangement. In real Python code, though, you usually do not want to manually write those steps every time. Python already has reliable sorting tools built in.

The first tool is list.sort(). This changes the original list in place. In place means the list object itself is rearranged. If scores = [3, 9, 1, 7] and you call scores.sort(), the variable scores now refers to [1, 3, 7, 9]. Nothing new has to be assigned because the existing list was modified.

The second tool is sorted(). This creates a new sorted list and leaves the original alone. That difference is small but important. If you need to keep the original order for later, sorted() is safer. If you want to permanently reorder the list you already have, sort() is direct and clean.

The third tool is reverse=True. By default, Python sorts from low to high or A to Z. reverse=True flips the result so numbers go high to low and strings go Z to A. The fourth tool, key=, tells Python what part of each item should be used for sorting. This is how you sort pairs by score, dictionaries by a field, or names by length instead of alphabetical order.

Python Sorting Tool Comparison

Tool	What it does	Best use
list.sort()	Reorders the original list in place.	Use when you want the existing list changed.
sorted(list)	Returns a new sorted list without changing the original.	Use when you need both original and sorted versions.
reverse=True	Flips the sorted order from ascending to descending.	Use for high-to-low scores, newest-to-oldest dates, or Z-to-A text.
key=	Tells Python what part of each item to sort by.	Use for tuples, dictionaries, objects, lengths, names, or custom rules.
lambda	A tiny unnamed function often used with key=.	Use when the sorting rule is simple and does not need a separate function.

Code Example - sort(), sorted(), and reverse=True

```
scores = [3, 9, 1, 7]

scores.sort()
print(scores)          # [1, 3, 7, 9]

original = [3, 9, 1, 7]
new_scores = sorted(original)
print(original)        # [3, 9, 1, 7]
print(new_scores)      # [1, 3, 7, 9]

scores.sort(reverse=True)
print(scores)          # [9, 7, 3, 1]
```

Plain English: sort() changes the list you already have. sorted() gives you a new list. reverse=True means descending order instead of ascending order.

Out Loud Programmer Thinking

Before I choose a sorting tool, I ask two questions: Do I want to change this exact list, or do I need a separate sorted copy? Then I ask what Python should compare for each item. If the answer is the whole item, the basic sort is enough. If the answer is one field or one position inside each item, I need key=.

That habit keeps the code from becoming random incantation soup. The line of code should match the sentence: sort this data by this piece, in this direction.

Made with Goodnotes

Lesson 13 - Using Python Sorting Tools in Practice (continued)

Code Example - Sorting Pairs with key=lambda

```
players = [("Alex", 80), ("Bob", 50), ("Cara", 90)]

players.sort(key=lambda pair: pair[1])
print(players)
# [("Bob", 50), ("Alex", 80), ("Cara", 90)]

players.sort(key=lambda pair: pair[1], reverse=True)
print(players)
# [("Cara", 90), ("Alex", 80), ("Bob", 50)]
```

Plain English: pair[1] means sort by the second value inside each pair. In this example, that second value is the score. Adding reverse=True puts the highest scores first.

Terms, Definitions, and Why They Are Called That

- In-place: changing the existing object instead of making a separate copy. list.sort() sorts in place because the original list is rearranged.
- Return value: the value a function gives back. sorted() returns a new list; list.sort() returns None because its main job is modifying the list itself.
- Ascending order: smallest to largest or A to Z. It is called ascending because the values climb upward.
- Descending order: largest to smallest or Z to A. It is called descending because the values move downward.
- Key function: a function that extracts the value Python should compare. key=lambda pair: pair[1] says, "Compare by the second item."
- Lambda: a small unnamed function. It is useful when the sorting rule is short enough to write right where it is used.

Common Mistakes and Helpful Tips

- Mistake: expecting scores.sort() to give back a new list. It does not. It changes scores directly and returns None.
- Mistake: using sorted() when you meant to permanently change the original list. sorted() gives a new list, so assign the result if you need it.
- Mistake: sorting pairs without a key and wondering why Python sorts by the first item. If you need the second item, use key=lambda pair: pair[1].
- Mistake: forgetting reverse=True when the question asks for largest first, highest score, most recent date, or descending order.
- Tip: before writing the sort line, say out loud: what exactly am I comparing? The whole item, the first value, the second value, a dictionary field, or a calculated value?

Mini Practice

Question	Best answer / thinking
You need scores from highest to lowest.	scores.sort(reverse=True) or sorted(scores, reverse=True).
You need a new sorted copy but must keep the old order.	Use sorted(original).
You have pairs like (name, score) and need score order.	Use key=lambda pair: pair[1].
You have names and need shortest to longest.	Use key=len or key=lambda name: len(name).

Notes

Made with Goodnotes

Code Example

Python Run Submit code

```python
scores = [3, 9, 1, 7]
scores.sort(reverse=True)
print(scores)
```

Output

```
[9, 7, 3, 1]
```

Python Run Submit code

```python
players = [
    ("Alice", 30),
    ("Bob", 50),
    ("Cara", 40),
]

players.sort(key=lambda pair: pair[1])
print(players)
```

Output

```
[('Alice', 30), ('Cara', 40), ('Bob', 50)]
```

Python Run Submit code

```python
players = [
    ("Alice", 30),
    ("Bob", 50),
    ("Cara", 40),
]

players.sort(key=lambda pair: pair[1], reverse=True)
print(players)
```

Output

```
[('Bob', 50), ('Cara', 40), ('Alice', 30)]
```

Python Run Submit code

```python
temps = [
    ("Denver", 55),
    ("Miami", 85),
    ("Seattle", 60),
    ("Chicago", 45),
]

temps.sort(key=lambda pair: pair[1], reverse=True)
print(temps)
```

Output

```
[('Miami', 85), ('Seattle', 60), ('Denver', 55),
('Chicago', 45)]
```

Python Run Submit code

```python
temps = [
    ("Denver", 55),
    ("Miami", 85),
    ("Seattle", 60),
    ("Chicago", 45),
]

temps.sort(key=lambda pair: pair[1])
print(temps)
```

Output

```
[('Chicago', 45), ('Denver', 55), ('Seattle', 60),
('Miami', 85)]
```

Made with Goodnotes

Lesson 14 - Advanced Sorting with Python Sorting Tools

Big Idea

Advanced sorting means sorting more realistic data: dictionaries, records, repeated values, ties, and multiple priorities. Instead of only sorting a list of numbers, you may sort players by score, sessions by client, photos by date, or tasks by deadline and priority.

The important thinking shift is this: the item being sorted may be complex, but the sort still needs one comparison rule. The key function is how you tell Python which part of the item matters first.

Full Teaching Explanation

A beginner sorting example usually looks like [4, 1, 8, 2]. Real programs are rarely that tidy. A photography workflow might have sessions with client names, dates, galleries, and delivery status. A game leaderboard might have player names and scores. A course gradebook might have students, assignments, and averages. The data is not a single number anymore; each item is a record.

Python handles this with key functions. If each item is a dictionary, the key function can pull out one field. For example, key=lambda player: player["score"] tells Python to compare the score field. The dictionaries stay intact, but the sort order is decided by that selected value.

Advanced sorting also includes ties. If two players both have a score of 200, sorting only by score does not fully explain which one should appear first. A multi-key sort uses a tuple as the key, such as (-score, name). Python compares the first part first. If the first parts tie, it uses the second part. That is how you can sort by highest score and then alphabetically by name.

The most useful question is: what is the primary sorting rule, and what is the backup rule if there is a tie? Once you can answer that, the Python code becomes much less mysterious.

Advanced Sorting Decision Guide

Situation	Sorting rule	Python idea
Players on a leaderboard	Highest score first, then name A-Z.	key=lambda p: (-p["score"], p["name"])
Client sessions	Client name A-Z, then date.	key=lambda s: (s["client"], s["date"])
Names by length	Shortest name first.	key=len or key=lambda n: len(n)
Tasks by priority	High priority first, then due date.	Use a key tuple with priority order and date.
Photos by timestamp	Oldest first or newest first.	Use key=date field and reverse=True when newest first.

Code Example - Sort Dictionaries by Score and Name

```
players = [
    {"name": "Alex",  "score": 120},
    {"name": "Brook", "score": 200},
    {"name": "Casey", "score": 200},
    {"name": "Drew",  "score": 150}
]

players.sort(key=lambda p: (-p["score"], p["name"]))
print(players)
```

Plain English: sort by negative score first so larger scores come earlier. If two scores tie, sort those tied players by name from A to Z.

Why Not Just Use reverse=True?

reverse=True reverses the entire final order. That is fine when every sorting priority should go backward. But if the problem says highest score first and names A-Z for ties, reversing the whole sort would also reverse the names. Using -score keeps score descending while leaving the name tie-breaker ascending.

Translation: reverse=True is great for one direction. Negative numeric keys are often better when only the number part should go high-to-low.

Made with Goodnotes

Lesson 14 - Advanced Sorting with Python Sorting Tools (continued)

Line-by-Line Thinking

Code piece	Plain-English meaning	Why it matters
p	One player dictionary at a time.	The lambda receives each item from the list.
p["score"]	The score field inside that player record.	This is the primary value we care about.
-p["score"]	A negative score.	Turns high scores into smaller comparison values, so high scores appear first in normal ascending sort.
p["name"]	The player name.	Used as the backup rule when scores tie.
(-p["score"], p["name"])	A tuple key with two priorities.	Python compares score first, then name only if needed.

Code Example - Sort Sessions by Client, Then Date

```
sessions = [
    {"client": "Rivera", "date": "2026-07-01"},
    {"client": "Adams",  "date": "2026-06-15"},
    {"client": "Rivera", "date": "2026-06-21"},
    {"client": "Bennett", "date": "2026-06-30"}
]

sessions.sort(key=lambda s: (s["client"], s["date"]))
print(sessions)
```

Plain English: group sessions by client name first. If the same client appears more than once, put that client's sessions in date order.

Terms, Definitions, and Why They Are Called That

- Record: one complete item with multiple pieces of information, such as a player with a name and score. Records act like rows in a table.
- Field: one named piece of information inside a record, such as score, client, date, or name. It is the specific part you may sort by.
- Primary key: the first sorting priority. In a leaderboard, score is usually the primary key.
- Secondary key: the backup sorting priority used when the primary key ties. In a leaderboard, name may be the secondary key.
- Tuple key: a key made of multiple values, like (-score, name). Python compares tuple items from left to right.
- Tie-breaker: the rule used when two items match on the main sorting value.

Common Mistakes and Helpful Tips

- Mistake: trying to sort dictionaries without telling Python which field matters. Dictionaries are complex records, so use key= to choose a field.
- Mistake: using reverse=True when only one part of a multi-key sort should be reversed. For high score then A-Z name, use negative score instead of reverse=True on the whole tuple.
- Mistake: forgetting that tuple keys compare left to right. The first value is the main rule; later values only matter when earlier values tie.
- Tip: write the sorting sentence before code: "Sort by score high to low, then name A-Z." Then convert each phrase into the key tuple.
- Tip: for advanced sorting, sketch one example tie. If two items have the same main value, what should happen next?

Notes

Made with Goodnotes

Code Example

Python

Run

Submit code

```python
players = [
    {"name": "Alex", "score": 120},
    {"name": "Brooke", "score": 200},
    {"name": "Casey", "score": 200},
    {"name": "Drew", "score": 150},
]

players.sort(key=lambda p: (-p["score"], p["name"]))

print(players)
```

Output

```
[{'name': 'Brooke', 'score': 200}, {'name': 'Casey',
'score': 200}, {'name': 'Drew', 'score': 150}, {'name':
'Alex', 'score': 120}]
```

Python

Run

Submit code

```python
players = [
    {"name": "Alex", "score": 120},
    {"name": "Brooke", "score": 200},
    {"name": "Casey", "score": 200},
    {"name": "Drew", "score": 150},
]

players.sort(key=lambda p: p["name"])                # 1) by name
players.sort(key=lambda p: p["score"], reverse=True)  # 2) by score
print(players)
```

Output

```
[{'name': 'Brooke', 'score': 200}, {'name': 'Casey', 'score': 200}, {'name': 'Drew',
'score': 150}, {'name': 'Alex', 'score': 120}]
```

Python

Run

Submit code

```python
books = [
    {"title": "A", "price": 12.99, "in_stock": True},
    {"title": "B", "price": 5.99, "in_stock": False},
    {"title": "C", "price": 7.50, "in_stock": True},
    {"title": "D", "price": 20.00, "in_stock": False},
]

books.sort(key=lambda b: (not b ["in_stock"], b["price"]))
print(books)
```

Output

```
[{'title': 'C', 'price': 7.5, 'in_stock': True}, {'title': 'A',
'price': 12.99, 'in_stock': True}, {'title': 'B', 'price': 5.99,
'in_stock': False}, {'title': 'D', 'price': 20.0, 'in_stock': False}]
```

Made with Goodnotes

Lesson 15 - Challenge: Advanced Sorting

Big Idea

The advanced sorting challenge asks you to choose and trace the right Python sorting tool for the situation. The answer is usually less about memorizing one line of code and more about reading the problem carefully: What data do I have? What field matters? Should the order be ascending or descending? What happens when values tie?

This is where Week 4 pulls together searching, sorting, efficiency, and Python syntax. You are proving that you can move from a story problem to a clean sorting rule.

Full Teaching Explanation

Advanced sorting problems can look intimidating because the data often has brackets, parentheses, dictionaries, strings, numbers, and lambda functions all piled together like a tiny syntax lasagna. The way through is to separate the data from the rule.

First, identify the shape of the data. A simple list like [7, 2, 9] can be sorted directly. A list of tuples like [("Alex", 80), ("Bob", 90)] needs you to decide whether the first value or second value matters. A list of dictionaries needs a field name, such as "score" or "client". Once you know the shape, the key function becomes a map from item to comparison value.

Second, identify the requested order. Words like smallest, earliest, A-Z, and increasing usually mean ascending order. Words like largest, highest, newest, Z-A, and decreasing usually mean descending order. For one sorting value, reverse=True is often enough. For multiple sorting values with mixed directions, a tuple key may be cleaner.

Third, check for ties. Tie rules are where many quiz mistakes happen. If the problem says sort by score descending, then by name ascending, you cannot just reverse the whole sort unless you are okay with names also going backward. A key like (-score, name) handles that mixed direction correctly.

Challenge Reading Checklist

Question to ask	What it tells you
What is each item?	Number, string, tuple, dictionary, or object.
What part should be compared?	Whole item, index 0, index 1, a dictionary field, length, date, or score.
Ascending or descending?	Whether you need normal order, reverse=True, a negative number, or another transformation.
Are there ties?	Whether you need a second or third sorting priority.
Should the original list change?	Use sort() for in-place change or sorted() for a new list.
What output is being asked for?	Final sorted list, one chosen line of code, printed output, or explanation.

Challenge Example - Choose the Correct Key

```
players = [
    {"name": "Maya", "score": 95},
    {"name": "Noah", "score": 88},
    {"name": "Zoe",  "score": 95},
    {"name": "Ari",  "score": 95}
]

players.sort(key=lambda p: (-p["score"], p["name"]))
print(players)
```

Plain English: score is the main rule, and high scores should come first. The negative score handles that. When scores tie, name decides the order: Ari, Maya, then Zoe.

Made with Goodnotes

Lesson 15 - Challenge: Advanced Sorting (continued)

Trace the Output

Original item	Key value	Why it lands there
{"name": "Maya", "score": 95}	(-95, "Maya")	Tied score group; M comes after A.
{"name": "Noah", "score": 88}	(-88, "Noah")	Lower score, so it comes after the 95 group.
{"name": "Zoe", "score": 95}	(-95, "Zoe")	Tied score group; Z comes last alphabetically.
{"name": "Ari", "score": 95}	(-95, "Ari")	Tied score group; A comes first alphabetically.

Final order: Ari, Maya, Zoe, Noah. The key values explain the order better than staring at the dictionaries and hoping the answer vibes its way into existence.

Common Advanced Sorting Challenge Types

Challenge type	How to handle it	Trap to avoid
Sort tuples by second value	Use key=lambda item: item[1].	Accidentally sorting by the first value.
Sort dictionaries by one field	Use key=lambda row: row["field"].	Forgetting the field name must be chosen.
Sort high to low	Use reverse=True for simple cases.	Using reverse=True when only one part of a tuple should reverse.
Sort by score descending, then name ascending	Use key=lambda p: (-p["score"], p["name"]).	Reversing both score and name.
Keep original list unchanged	Use sorted(data) and assign the new list.	Using data.sort() and losing the old order.
Explain output	Write the key value beside each item.	Trying to mentally sort complex records without tracing.

Week 4 Connection - Searching After Sorting

Sorting and searching often work together. If data is small or unsorted and you only search once, linear search may be perfectly reasonable. If data is large and you will search it repeatedly, sorting first can make faster search strategies possible.

The important trade-off is setup cost versus repeated benefit. Sorting takes work upfront. Binary search benefits from that organization later. That is the same theme the whole week has been building: algorithms are not just about getting an answer; they are about choosing a strategy that fits the situation.

Final Checklist for Advanced Sorting Questions

- Circle the data shape: list, tuple, dictionary, nested list, or record.
- Underline the sorting rule: score, name, date, length, second item, or another field.
- Mark the order words: highest, lowest, newest, oldest, A-Z, Z-A, increasing, or decreasing.
- Look for tie-breakers. If the problem says "then by," you probably need a tuple key.
- Decide whether the original list should change. If yes, use sort(). If no, use sorted().
- For mixed directions, avoid blindly using reverse=True. Consider negative numbers for descending numeric fields.
- Trace the key value beside each item when the output feels confusing.

Notes

Made with Goodnotes

Week 4 Review Tools - Searching and Sorting

Big Picture Review

Week 4 ties together two huge programming habits: finding information and arranging information. Searching asks, "Where is the thing I want?" Sorting asks, "What useful order should this data be in?" The skills work together because the way data is organized changes how easily it can be searched.

The main search contrast is linear search versus binary search. Linear search is the steady, reliable option: check one item at a time. Binary search is the fast option when the list is sorted: look in the middle, throw away half the possibilities, and repeat. The main sorting contrast is manual algorithm thinking versus Python tool thinking. Bubble sort and insertion sort teach the movement logic. Python sort(), sorted(), key=, lambda, and reverse=True are the practical tools you will usually use in real code.

Week 4 Concept Map

Concept area	Main idea	Refresher sentence
Search target	The exact value you want to find.	If you cannot name the target, you cannot design the search clearly.
Search space	The collection or area you are allowed to search.	The search space is the pile, list, folder, or dataset where the target may live.
Linear search	Check each item one by one.	Best for small, unsorted, simple data.
Binary search	Use the middle of a sorted list to cut possibilities in half.	Fast, but only valid when sorted order is guaranteed.
Sorting	Arrange data into a useful order.	Sorting can make output clearer and searching faster.
Python sorting tools	Use sort(), sorted(), key=, lambda, and reverse=True.	Use tools when the goal is real work, not practicing the algorithm by hand.

Memory Hooks

- Linear search = walk down the line and ask, "Is this it?"
- Binary search = sorted list only, then check the middle and cut the problem in half.
- Bubble sort = neighbors swap until large values bubble toward the end.
- Insertion sort = take one item and slide it left into the sorted part.
- sort() changes the original list; sorted() makes a new sorted list.
- key= answers the question, "What part of each item should Python compare?"

Notes

Made with Goodnotes

Search Review - Targets, Spaces, and Comparisons

Full Teaching Review

Every search problem has three basic pieces: a target, a search space, and comparison steps. The target is the thing you want. The search space is where you are allowed to look. A comparison step is the moment the program asks, "Is this current item the target?"

This vocabulary matters because it keeps search problems from feeling like vague scavenger hunts. If the problem says to find 9 in nums = [3, 7, 9, 2], the target is 9, the search space is the list nums, and each comparison is one check against an item in the list. The same logic applies to contacts, emails, photos, files, or customer records.

When tracing search code, write each comparison in order. Do not jump straight to the answer. Search questions often ask what happens step by step, what gets printed, what index is returned, or how many comparisons happen before the target is found or the search fails.

Search Vocabulary Table

Term	Definition	Why the name makes sense
Search target	The value being searched for.	It is the target because the whole search is aimed at finding it.
Search space	The collection being searched.	It is the allowed space where the target may be located.
Comparison	One check between the current item and the target.	The program compares candidate value versus target value.
Found case	The search reaches the target.	The algorithm can stop because the job is complete.
Not found case	The search finishes without a match.	The algorithm must report failure instead of pretending it found something.
Index	The position where a target is found.	The index answers "where" inside the list.

Linear Search Trace Pattern

```
nums = [3, 7, 9, 2]
target = 9

for i in range(len(nums)):
    if nums[i] == target:
        print("Found at index", i)
        break
```

Plain English: start at the beginning, check each item against the target, and stop as soon as the target is found.

Trace It Out Loud

- Check index 0: nums[0] is 3. Is 3 equal to 9? No.
- Check index 1: nums[1] is 7. Is 7 equal to 9? No.
- Check index 2: nums[2] is 9. Is 9 equal to 9? Yes.
- Print "Found at index 2" and stop the loop.

Made with Goodnotes

Linear Search vs Binary Search Decision Guide

Full Teaching Review

Linear search is the safest search pattern because it does not need the data to be sorted. It simply checks candidates one by one. That makes it readable and reliable, especially when the list is small, unsorted, changing constantly, or only searched once.

Binary search is faster when the data is sorted because every comparison removes about half of the remaining search space. The danger is that binary search depends completely on order. If the list is not sorted, the middle-value decision becomes meaningless and the algorithm can eliminate the side where the target actually lives.

The best decision is not "binary is always better." The best decision is "what does this situation allow?" If the data is small or unsorted, linear search is often the right first thought. If the data is large and sorted, binary search becomes the stronger choice.

Search Algorithm Comparison

Question	Linear search	Binary search
Does data need to be sorted?	No. It works on sorted or unsorted data.	Yes. Sorted order is required.
How does it search?	Checks each item one at a time.	Checks the middle, then keeps only the half where the target could be.
Best for	Small lists, unsorted lists, one-time searches, simple code.	Large sorted lists, repeated searches, data where order is guaranteed.
Worst case	May check every item.	May keep halving until the search space is empty.
Main risk	Can be slow for large data.	Wrong answer risk if the data is not sorted.
Memory hook	Walk down the line.	Middle, half, repeat.

Binary Search Checklist

- Is the data sorted? If no, do not use binary search yet.
- What is the target?
- What are the low and high boundaries?
- What is the middle index?
- Is the middle value equal to the target, too small, or too large?
- Which half can be safely ignored?

Binary Search Boundary Pattern

```
low = 0
high = len(nums) - 1

while low <= high:
    mid = (low + high) // 2

    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
```

Plain English: keep a left boundary and right boundary. Check the middle. Move one boundary inward after each comparison.

Made with Goodnotes

Sorting Review - Why Sorting Helps

Full Teaching Review

Sorting is the act of arranging values into a useful order. That order might be alphabetical, numerical, chronological, highest-to-lowest, lowest-to-highest, newest-to-oldest, or grouped by a field like client name. Sorting does not just make data look nicer. It can make later work easier.

Sorted data is easier for humans because patterns become visible. A contact list sorted A-Z is easier to scan than names in random order. Photos sorted by session or date are easier to deliver. Scores sorted highest-to-lowest make rankings obvious. Sorted data can also help algorithms. Binary search, for example, only works because sorted order lets the algorithm know which side of the list to eliminate.

The trade-off is that sorting takes work. If the data changes constantly or is only needed once, sorting may not be worth the setup. If the same data will be searched, displayed, filtered, or compared many times, sorting can save frustration later. The Week 4 habit is to ask whether order is useful enough to justify the work.

Sorting Decision Questions

Question	Why it matters	Likely decision
Will humans read this output?	Humans understand ordered lists faster.	Sort if order improves readability.
Will the data be searched many times?	Repeated searching may benefit from sorted organization.	Consider sorting or using a faster lookup structure.
Does binary search need to be used?	Binary search requires sorted data.	Sort first or use already sorted data.
Does the data change constantly?	Repeated sorting can waste work.	Be careful; sorting every tiny update may be inefficient.
Is the list tiny?	The performance difference may not matter.	Use the simplest readable option.
Is the sort rule clear?	Unclear rules produce confusing output.	Define what field or value controls order first.

Sorting Mental Models

- Alphabetical contact list: sorting helps humans find names faster.
- Leaderboard: sorting makes rank visible immediately.
- Photo sessions: sorting by client/date makes delivery and retrieval less chaotic.
- Deck of cards: sorting creates structure from randomness.
- Search preparation: sorting can unlock binary search when the sorted order is valid.

Notes

Made with Goodnotes

Manual Sorting Algorithms - Bubble and Insertion

Full Teaching Review

Bubble sort and insertion sort are not usually the fastest production choices, but they are excellent teaching algorithms because they show how sorting can happen through repeated comparisons and movement. The point is not to worship bubble sort like it is a tiny algorithm celebrity. The point is to understand the movement pattern.

Bubble sort compares neighbors. If two neighboring values are in the wrong order, it swaps them. After one full pass, a large value has usually moved toward the end, like a bubble rising. Repeated passes keep pushing values into place. Bubble sort is easy to trace, but it repeats a lot of work.

Insertion sort grows a sorted section on the left. It takes the next unsorted item and inserts it into the correct spot among the already sorted values. This feels like sorting cards in your hand. You pick up a card, slide it left past larger cards, and place it where it belongs.

Bubble vs Insertion Sort

Feature	Bubble sort	Insertion sort
Core movement	Swap neighboring values that are out of order.	Take one item and slide it left into the sorted part.
Mental model	Bigger values bubble toward the end.	Sorting playing cards in your hand.
What it compares	Adjacent neighbors.	Current item against previous sorted items.
Good teaching purpose	Shows repeated neighbor comparisons and swaps.	Shows growing a sorted section.
Main weakness	Can repeat many passes and waste work.	Can still be slow if many values must shift.
Common beginner trap	Forgetting that one pass may not fully sort the list.	Losing the saved current value while shifting items.

Bubble Sort Shape

```
for pass_num in range(len(nums) - 1):
    for i in range(len(nums) - 1):
        if nums[i] > nums[i + 1]:
            nums[i], nums[i + 1] = nums[i + 1], nums[i]
```

Plain English: repeat passes through the list. On each pass, compare neighbors and swap if the left value is larger than the right value.

Insertion Sort Shape

```
for i in range(1, len(nums)):
    current = nums[i]
    j = i - 1

    while j >= 0 and nums[j] > current:
        nums[j + 1] = nums[j]
        j -= 1

    nums[j + 1] = current
```

Plain English: save the current value, slide larger sorted values to the right, then insert the saved value into the open spot.

Manual Sorting Quick Checks

- Bubble sort question? Look for neighboring values and swap decisions.
- Insertion sort question? Look for current value, sorted-left section, shifting, and final insertion.
- One pass is not always a full sort. Pay attention to whether the question asks for one pass or the final result.
- If the list changes after each operation, write the list state. Your eyes are not a runtime environment, tragically.

Made with Goodnotes

Python Sorting Tools Review

Full Teaching Review

Manual algorithms explain how sorting works. Python sorting tools are what you use when you need practical, readable code. The two biggest tools are list.sort() and sorted(). Both arrange data, but they do not behave the same way.

list.sort() changes the original list in place. That means the existing list is rearranged. sorted() returns a new sorted list and leaves the original list alone. This distinction appears constantly in quiz questions because beginners often expect sort() to return a new list. It does not. If you write result = scores.sort(), result becomes None, because sort() modifies scores directly.

The key= argument tells Python what to compare. With simple numbers, the whole item is the comparison value. With complex items like tuples or dictionaries, you often need to point Python to the score, name, date, length, or another field. lambda is a quick way to write that extraction rule in place.

Python Sorting Tool Table

Tool	What it does	Common use
list.sort()	Sorts the original list in place and returns None.	When you want to permanently rearrange the list.
sorted(data)	Creates and returns a new sorted list.	When you want a sorted copy and still need the original order.
reverse=True	Sorts in descending order for the whole key.	Highest score first, newest date first, Z-A text.
key=len	Sorts strings or collections by length.	Shortest to longest names, words, or lists.
key=lambda item: item[1]	Sorts by a selected part of each item.	Sort pairs by score, price, date, or second field.
key tuple	Sorts by multiple priorities.	Sort by score first, then name if scores tie.

sort() vs sorted() Check

```
scores = [80, 95, 70]

changed = scores.sort()
print(scores)    # [70, 80, 95]
print(changed)   # None

original = [80, 95, 70]
copy = sorted(original)
print(original)  # [80, 95, 70]
print(copy)      # [70, 80, 95]
```

Plain English: sort() changes the list and returns None. sorted() returns a new list and leaves the old one alone.

Common Sorting Tool Mistakes

- Mistake: assigning result = scores.sort() and expecting result to be the sorted list.
- Mistake: using reverse=True when only one part of a multi-part key should go backward.
- Mistake: forgetting that key= chooses what to compare, not what to print.
- Mistake: sorting dictionaries without pointing to a specific field.
- Tip: say the rule in English first: sort players by score descending, then name ascending.

Made with Goodnotes

Trace Practice - Search and Sort

How to Use This Page

Tracing means acting like the computer. Do not summarize. Do not skip. Write the changing values after each important step. For search, track the current item, comparison, boundaries, and result. For sorting, track list state after swaps, shifts, or passes.

The goal is not speed at first. The goal is correctness. Once you can trace slowly, the patterns become familiar and faster. This is where the quiz goblins lose their power. Dramatic, but true.

Practice Set

Prompt	Trace focus	Answer / expected thinking
Linear search: find 4 in [8, 2, 4, 9].	Comparison order.	Check 8 no, 2 no, 4 yes. Found at index 2.
Linear search: find 7 in [1, 3, 5].	Not found case.	Check every item. No match, so return -1, False, or "not found" depending on code.
Binary search: find 11 in [3, 5, 7, 11, 15].	Middle and boundaries.	Middle 7 too small, search right half. Next middle 11 found.
Binary search on [9, 1, 5, 3].	Prerequisite check.	Invalid choice unless sorted first. Binary search requires sorted data.
Bubble one pass on [4, 2, 3].	Neighbor swaps.	Compare 4 and 2 swap -> [2,4,3]. Compare 4 and 3 swap -> [2,3,4].
Insertion sort first insert in [5, 2, 4].	Shifting.	Save 2. Shift 5 right. Insert 2 at front -> [2,5,4].

Trace Checklist

- For linear search: write each item checked and whether it matches.
- For binary search: write low, high, mid, middle value, and which boundary changes.
- For bubble sort: write the list after every swap or after each full pass, depending on the question.
- For insertion sort: write the current value, shifted values, and where current gets inserted.
- For Python sort tools: identify whether the original list changes or a new list is created.

Notes

Made with Goodnotes

Weekly Review Question Breakdown

How to Read Week 4 Questions

Most Week 4 review questions are not trying to trick you with brand-new ideas. They usually test one of five things: recognizing the search type, checking the sorted-data requirement, tracing loop behavior, choosing the correct sorting tool, or understanding what key= is doing.

The quickest way to slow down correctly is to mark the clue words. Words like unsorted, small, one by one, every item, middle, sorted, half, highest, lowest, alphabetical, in place, copy, field, and tie usually point directly to the concept being tested.

Question Clue Decoder

Clue in the question	Likely concept	What to do
"Check each item" or "one by one"	Linear search	Trace candidates from start to finish.
"Sorted list" and "middle"	Binary search	Track low, high, mid, and boundary changes.
"Unsorted list"	Binary search warning	Do not choose binary search unless sorting happens first.
"Neighbors" and "swap"	Bubble sort	Compare adjacent values and move large values right.
"Insert into sorted part"	Insertion sort	Track current value and shifting.
"Change original list"	list.sort()	Use sort(); remember it returns None.
"Keep original order"	sorted()	Use sorted() and assign the new list.
"Sort by score/name/date"	key=	Extract the field Python should compare.
"Then by"	Tuple key	Use multiple priorities in order.

Answer Strategy

- Step 1: Identify whether the problem is asking about searching or sorting.
- Step 2: Circle the condition words: sorted, unsorted, small, huge, repeated, first, highest, by score, by name.
- Step 3: Decide whether you need an algorithm idea or a Python tool.
- Step 4: Trace only the variables that matter. Do not rewrite the whole program unless needed.
- Step 5: Check whether the answer should be an index, a Boolean, a modified list, a new list, or printed output.

Mini Self-Test

Question	Answer
What search works on unsorted data?	Linear search.
What search needs sorted data?	Binary search.
What sorting algorithm swaps neighbors?	Bubble sort.
What sorting algorithm inserts into an already sorted section?	Insertion sort.
What Python tool leaves the original list unchanged?	sorted().
What argument tells Python what field to sort by?	key=.

Made with Goodnotes

Common Exam Traps and Debug Thinking

Trap Review

Week 4 has a few traps that show up because the concepts sound similar. Searching and sorting are related, but they are not the same. Linear and binary search both find targets, but they have different requirements. sort() and sorted() both arrange data, but one changes the original list and one creates a new list.

When you feel stuck, pause and ask what the code is trying to protect: the target, the order, the boundaries, the original list, or the selected sorting field. That usually reveals the mistake.

Mistake Fix Table

Mistake	Why it is wrong	Fix
Using binary search on an unsorted list.	The middle comparison only makes sense when order is guaranteed.	Sort first or use linear search.
Forgetting to update low or high.	The loop may repeat forever or search the wrong range.	After each comparison, move the correct boundary.
Using = instead of == in a comparison.	= assigns; == compares.	Use == when asking whether two values match.
Expecting sort() to return a sorted list.	sort() changes the original list and returns None.	Use scores.sort() alone or use sorted(scores).
Using reverse=True for mixed direction sorting.	It reverses the whole result, including tie-breakers.	Use a tuple key like (-score, name).
Not saving the current value in insertion sort.	Shifting can overwrite the value you are trying to insert.	Store current before the while loop.
Thinking one bubble pass always fully sorts.	One pass may only move one large value into place.	Continue passes until the list is sorted or no swaps happen.

Debug Questions to Ask Yourself

- What exact target am I searching for?
- Is the search space sorted or unsorted?
- What line performs the comparison?
- What changes after each loop: an index, a boundary, a list order, or a sorted copy?
- Am I supposed to return an index, print a message, return True/False, or return a sorted list?
- Does this function mutate the original list or create a new one?

Notes

Made with Goodnotes

Week 4 Cheat Sheet

Searching Cheat Sheet

Pattern	Use when	Core logic
Linear search	Small or unsorted data; simple one-time search.	for each item: compare item to target.
Binary search	Large sorted data.	check middle; remove half; repeat.
Found target	Current item equals target.	Return index, True, or found message.
Not found	Search finishes without a match.	Return -1, False, None, or not found message.
Search trace	Quiz asks what happens step by step.	Write each comparison and state change.

Sorting Cheat Sheet

Pattern/tool	Use when	Core logic
Bubble sort	Practicing neighbor comparison and swaps.	Compare adjacent values; swap if wrong order.
Insertion sort	Practicing sorted-section growth.	Take current item; slide bigger values; insert current.
list.sort()	Original list should change.	Mutates list in place; returns None.
sorted(data)	Need a new sorted copy.	Returns a new list; original stays unchanged.
reverse=True	Whole result should be descending.	Flips sort order.
key=	Need to sort by a field or custom rule.	Extracts the comparison value for each item.
tuple key	Need tie-breakers.	Sorts by first value, then second, then third.

Most Useful Python Sorting Patterns

```
nums.sort()
sorted_nums = sorted(nums)
nums.sort(reverse=True)

names.sort(key=len)
players.sort(key=lambda p: p["score"])
players.sort(key=lambda p: (-p["score"], p["name"]))
```

Plain English: choose sort() or sorted() first, then add reverse= or key= only when the problem asks for direction or a specific field.

One-Minute Review

- Linear search is flexible but may check every item.
- Binary search is fast but requires sorted data.
- Sorting creates order, but sorting itself costs work.
- Bubble sort swaps neighbors; insertion sort inserts into a sorted section.
- Python sorting tools are practical; manual sorting algorithms are mostly for understanding and tracing.
- key= is the sorting rule. reverse=True is the direction.

Notes

Made with Goodnotes

Final Week 4 Memory Sheet

What You Should Be Able to Explain

- Explain what a search target is and what a search space is.
- Trace a basic linear search and identify the comparison step.
- Explain why binary search only works on sorted data.
- Trace low, high, and mid through a simple binary search.
- Explain why sorting can make later searching or human reading easier.
- Describe the movement pattern of bubble sort and insertion sort.
- Choose between sort() and sorted() based on whether the original list should change.
- Use key= to sort by a selected field or value.
- Use a tuple key when the problem has tie-breakers.
- Recognize the common traps before they eat your quiz score for breakfast.

Final Plain-English Summary

Searching is about finding a target inside a search space. Linear search checks one item at a time and works almost anywhere. Binary search checks the middle and cuts the search space in half, but it only works when the data is sorted.

Sorting is about arranging data into a useful order. Bubble sort and insertion sort teach how values move through comparison, swapping, and shifting. Python sorting tools let you apply those ideas cleanly in real programs with sort(), sorted(), reverse=True, key=, lambda, and tuple keys.

The deepest Week 4 skill is decision-making. Do not just ask, "What algorithm do I know?" Ask, "What does this data look like? What does the problem require? Does order matter? Will I search once or many times? Should the original list change? What field controls the order?" That is programmer thinking, not just syntax survival.

Last-Minute Memory Hooks

- Target + search space + comparison = search thinking.
- Unsorted? Linear is safe. Sorted and large? Binary becomes powerful.
- Binary search without sorting is like using a map after someone shuffled the streets. Bold choice. Bad choice, but bold.
- Bubble sort swaps neighbors. Insertion sort slides values into place.
- sort() mutates. sorted() creates. key= compares. reverse=True flips.
- When confused, trace slowly. The computer is not guessing, and neither should we.

Notes

Made with Goodnotes